Installation Elastic et déploiement des agents

1. Mise en place d'un environnement Elastic Security

On commence par récupérer le dépôt git :

```
root@debian13vm:~# git clone https://github.com/pushou/siem.git
Clonage dans 'siem'...
remote: Enumerating objects: 627, done.
remote: Counting objects: 100% (49/49), done.
remote: Compressing objects: 100% (25/25), done.
remote: Total 627 (delta 32), reused 37 (delta 24), pack-reused 578 (from 1)
Réception d'objets: 100% (627/627), 33.99 Mio | 7.21 Mio/s, fait.
Résolution des deltas: 100% (344/344), fait.
root@debian13vm:~#
```

Ensuite on rajoute **vm.max_map_count=262144** à **/etc/sysctl.conf** :

```
  GNU nano 8.4                                          /etc/sysctl.conf
vm.max_map_count=262144
```

On utilise ensuite la commande : **sysctl -p** pour vérifier que la ligne s'est bien ajouté et est fonctionnelle.

```
root@debian13vm:~/siem# sysctl -p
vm.max_map_count = 262144
```

Ensuite on lance **make es** qui va venir nous construire les conteneurs :

```
rm -f /root/siem/.env
sudo chown -R 0:0 /root/siem
make[1] : on quitte le répertoire « /root/siem »
b7777fc2754f0db8090a5cc259c3c0cc0d4ecee3ce6ba4fae9ec1f4611ad5520
elasticdata
elasticconfig
certs
Unable to find image 'docker.elastic.co/elasticsearch/elasticsearch:9.1.5' locally
9.1.5: Pulling from elasticsearch/elasticsearch
6beba0d3d2b7: Pull complete
836fa7269e17: Pull complete
f05dbae21513: Pull complete
37c6084d18f1: Pull complete
4ca545ee6d5d: Pull complete
967bab14c0d5: Pull complete
b9d141a750c9: Pull complete
1e63b7ec60ae: Pull complete
80cc91dc4d69: Pull complete
2b419444db78: Pull complete
Digest: sha256:38604132e9d6cf4cd0acc090b3de645361dc142384e5341338aecb863741c630
Status: Downloaded newer image for docker.elastic.co/elasticsearch/elasticsearch:9.1.5
Creating CA
Archive:  config/certs/ca.zip
  inflating: config/certs/ca/ca.crt
  inflating: config/certs/ca/ca.key
Creating certs
Archive:  config/certs/certs.zip
   creating: config/certs/es01/
  inflating: config/certs/es01/es01.crt
  inflating: config/certs/es01/es01.key
   creating: config/certs/es02/
  inflating: config/certs/es02/es02.crt
  inflating: config/certs/es02/es02.key
   creating: config/certs/es03/
  inflating: config/certs/es03/es03.crt
  inflating: config/certs/es03/es03.key
   creating: config/certs/kibana/
  inflating: config/certs/kibana/kibana.crt
  inflating: config/certs/kibana/kibana.key
   creating: config/certs/fleet/
  inflating: config/certs/fleet/fleet.crt
  inflating: config/certs/fleet/fleet.key
Setting file permissions
All done!
396f046a8d73768969e46df463b0bf902adedcfaa1ce797c630358b047e80026
/root/siem/temp/ca.crt
```

De plus on retrouve les mots de passe générés

```
Successfully copied 3.07kB to /root/siem/temp/ca.crt
Attente ES up (peut prendre quelques minutes)...
génération des password pour les systèmes et sauvegarde dans  (pour docker-compose) et /root/siem/secrets/passwords.txt (pour docker run)
password elastic nxlJPulte=z2tBZKPqxM
password kibana_system rJATP5ZpwfA75kq0JM8b
password beats MvnGyS4eECfj9BLew2zj
password apm MvnGyS4eECfj9BLew2zj
password monitoring aHtm63AR8Zokg0UhNI=3
copy du certificat de l'AC sur votre hôte
Updating certificates in /etc/ssl/certs...
```

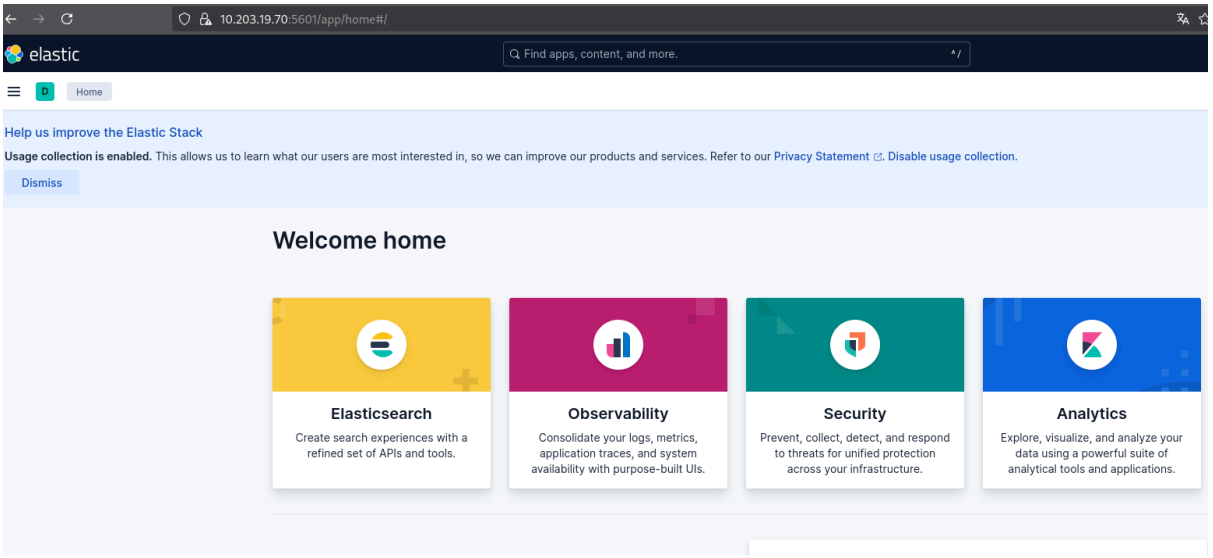
Pour installer l’Elastic Siem on utilise la commande **make siem**

```
Digest: sha256:1a4f4fdd13938ac2459aebb7080aac68b199334b8255653decdd38f8e38c7048
Status: Downloaded newer image for jasonish/suricata:latest
05a8558fe0878c5bedb3b36821adb3d9111ceb4e27fd2fbca60e7fccbedbfd8a
AlmaLinux 9 - AppStream                                                      2.1 MB/s |  13 MB     00:06
AlmaLinux 9 - BaseOS                                                         9.8 MB/s | 8.6 MB     00:00
AlmaLinux 9 - Extras                                                          35 kB/s |  20 kB     00:00
Extra Packages for Enterprise Linux 9 - x86_64                                10 MB/s |  20 MB     00:01
Extra Packages for Enterprise Linux 9 openh264 (From Cisco) - x86_64         909  B/s | 2.5 kB     00:02
Dependencies resolved.
==========================================================================================================
 Package                   Architecture         Version                  Repository                  Size
==========================================================================================================
Upgrading:
 epel-release              noarch               9-10.el9                 epel                        19 k
 gnupg2                    x86_64               2.3.3-5.el9_7            baseos                     2.5 M

Transaction Summary
==========================================================================================================
Upgrade  2 Packages

Total download size: 2.5 M
Downloading Packages:
Extra Packages for Enterprise Linux 9 - x86_64               464% [===========================================
==========================================================================================================
(1/2): epel-release-9-10.el9.noarch.rpm                                       634 kB/s |  19 kB     00:00
(2/2): gnupg2-2.3.3-5.el9_7.x86_64.rpm                                        5.7 MB/s | 2.5 MB     00:00
----------------------------------------------------------------------------------------------------------
```

Enfin, on fait la commande **make fleet**

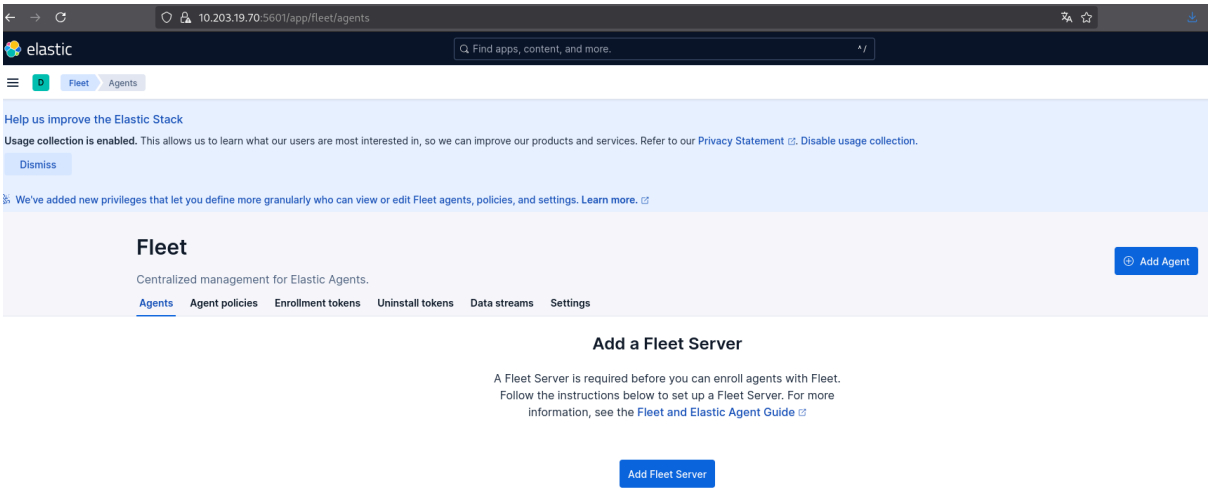
```
/root/siem/scripts/print_password.sh
password elastic= nxlJPulte=z2tBZKPqxM
password kibana= rJATP5ZpwfA75kq0JM8b
password beats_system= MvnGyS4eECfj9BLew2zj
password apm_system=  MvnGyS4eECfj9BLew2zj
password remote_monitoring_user= aHtm63AR8Zokg0UhNI=3
token fleet=
make[1] : on quitte le répertoire « /root/siem »
TEMP_DIR /root/siem/temp
CA_FILE /root/siem/temp/ca.crt
SECRETS_DIR /root/siem/secrets
CONFIG_DIR /root/siem/config
ETC_DIR /root/siem/etc
LOGS_DIR /root/siem/logs
PASSWORDS_FILE /root/siem/secrets/passwords.txt
IP_HOST 10.203.19.70
ELASTIC_PASSWORD nxlJPulte=z2tBZKPqxM
{"isInitialized":true,"nonFatalErrors":[]}{"item":{"id":"fleet-server-policy-jmp","name":"Fleet Server policy jmp","description":"","namespace":"defaul
ut":1209600,"status":"active","is_managed":false,"revision":1,"updated_at":"2026-01-22T09:47:15.234Z","updated_by":"elastic","schema_version":"1.1.1","
,"message":"[request body.fleet_server_hosts]: definition for this key is missing"}  % Total    % Received % Xferd  Average Speed   Time    Time     Ti
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0/root/siem/scripts/lance-fleet.sh: ligne 47: jq : commande introuvable
100   131  100   131    0     0    299      0 --:--:-- --:--:-- --:--:--   299
curl: Failed writing body
FLEET_TOKEN
lancement de fleet
Unable to find image 'docker.elastic.co/elastic-agent/elastic-agent:9.1.5' locally
9.1.5: Pulling from elastic-agent/elastic-agent
6beba0d3d2b7: Already exists
1cf0ca7ee29d: Pull complete
7948dada2fbb: Pull complete
29c9638869c2: Pull complete
c3946d6cba00: Pull complete
3e3f5af70af5: Pull complete
de137e607b92: Pull complete
fe88d4e14d35: Pull complete
46791c729233: Pull complete
15628fbbb84a: Pull complete
4ca545ee6d5d: Pull complete
Digest: sha256:c66590a91c67dd8218d9b35c3edf69ffd76be59e71481cf0da9584b9e256c6a2
Status: Downloaded newer image for docker.elastic.co/elastic-agent/elastic-agent:9.1.5
f2f92a5ec8ea636d5e100f35103edae9bccfb64ad35c4088470ce69073893256
```

Ensuite on se connecte à https://10.203.19.70:5601/ pour accéder à Elastic (username elastic ; password : Z_7=ce5MljI0LxKKpnIF)

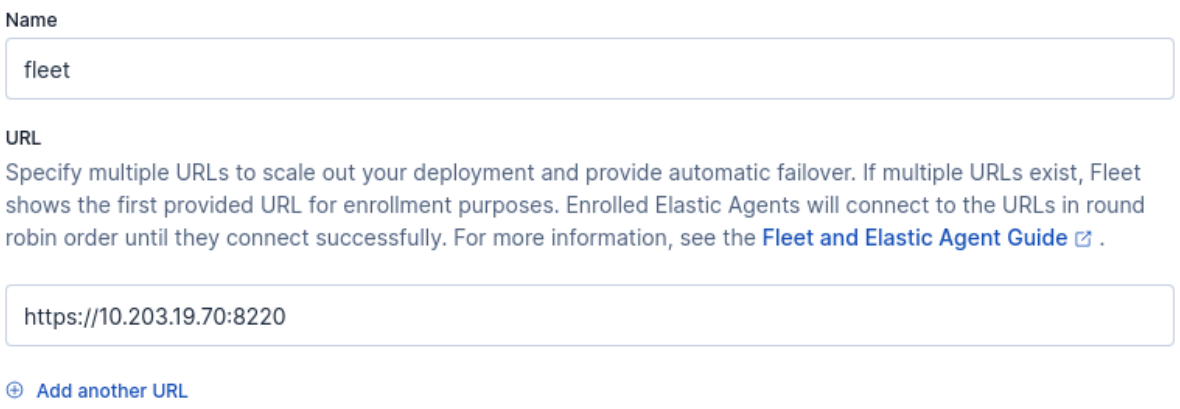


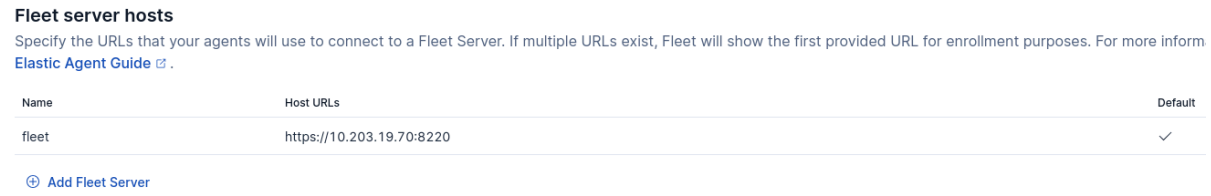
2. Configuration de fleet

On se rends dans **Management > Fleet** :



Puis dans Settings > Add Fleet Server et on met l'IP et le port 8220 :

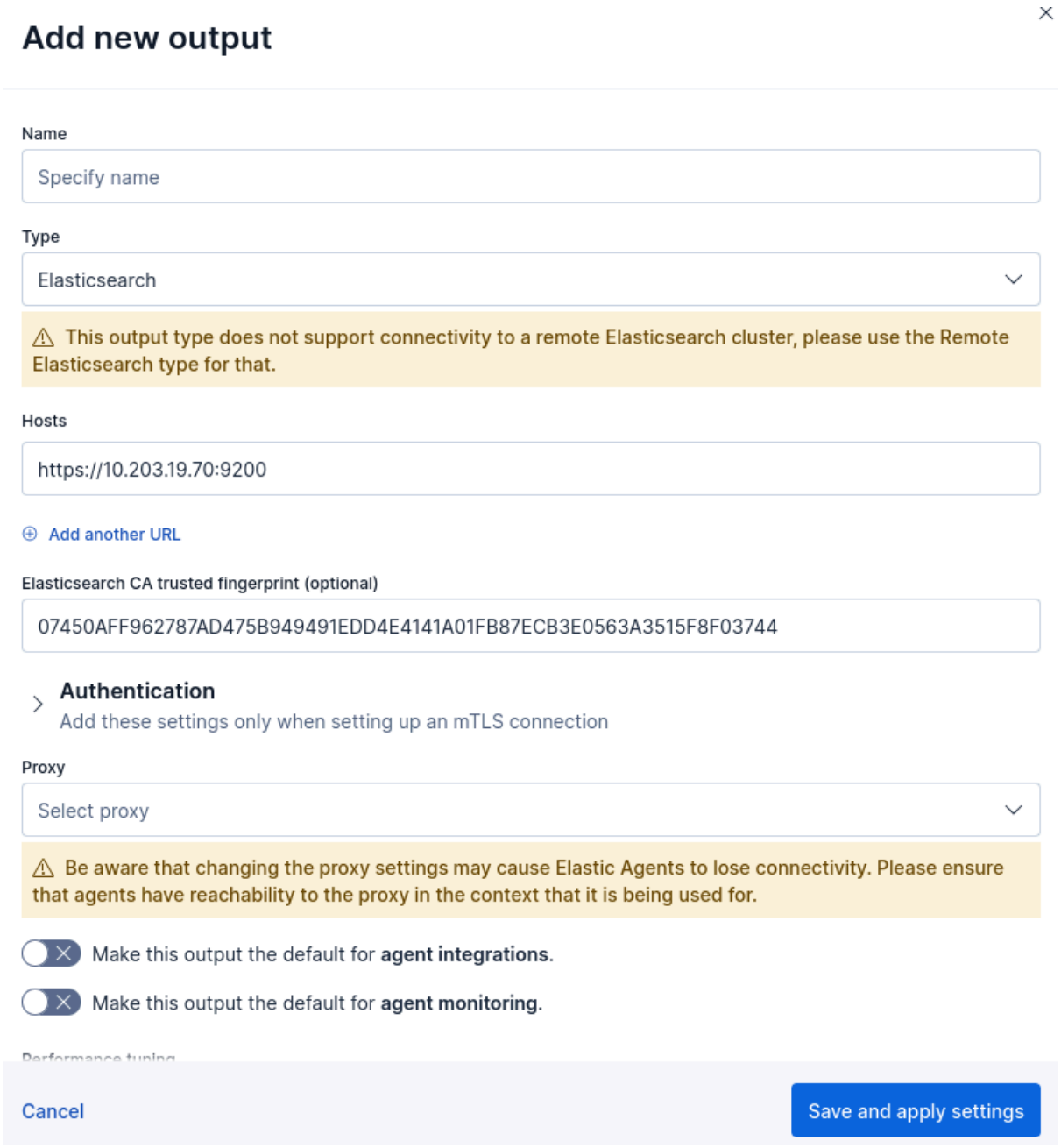




Afin de configurer les output de fleet on fait les commandes **make fgprint** et **make prca**

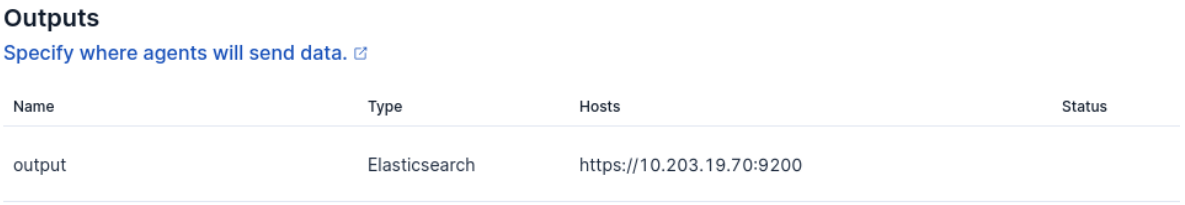
```
root@debian13vm:~/siem# make fgprint
/root/siem/scripts/getFingerprint.sh
07450AFF962787AD475B949491EDD4E4141A01FB87ECB3E0563A3515F8F03744
root@debian13vm:~/siem# make prca
/root/siem/scripts/printcrt.sh
ssl:
  certificate_authorities:
    - |
      -----BEGIN CERTIFICATE-----
      MIIDWjCCAkKgAwIBAgIVAN90Mp8rCIOSsjr/Aep4G6UPukWmMA0GCSqGSIb3DQEB
      CwUAMDQxMjAwBgNVBAMTKUVsYXN0aWMgQ2VydGlmaWNhdGUgVG9vbCBBdXRvZ2Vu
      ZXJhdGVkIENBMB4XDTI2MDEyMjA5MzE0NFoXDTI5MDEyMTA5MzE0NFowNDEyMDAG
      A1UEAxMpRWxhc3RpYyBDZXJ0aWZpY2F0ZSBUb29sIEF1dG9nZW5lcmF0ZWQgQ0Ew
      ggEiMA0GCSqGSIb3DQEBAQUAA4IBDwAwggEKAoIBAQC8jsgEhJAyrakquiMBFMlV
      Q6S5qimJRLB1GV3dIr8l7v34LLtwhYzRPYZXX/aLvHqi+/opnxzv5UvVEblErD00
      5Vo0GCVQL7KeEIFf9SSW0NwG+BLk3thEtchfWQA5MZZaGQRjhJFYsi5SmY38vBD+
      JfBddPC7cPUJ4A4qpy9vPtHKd6heb/t8CYvD+zpJhhu+WFazM61sxNfLXEMHhHJn
      hMrYonakg/KvByk7m6ybr++X+O+VffnrBSE9KsdSFnlRHslq7t8YxNziKrPeQerU
      XNsQMR3NL+KB3+QiU+Cafs6cbf7mbHhpLgLwYrso5oRMS/q/crBx1XM1mG6U6x/H
      AgMBAAGjYzBhMB0GA1UdDgQWBBSfzLNAEM0etwdRIbz/BPr32LsBVDAfBgNVHSME
      GDAWgBSfzLNAEM0etwdRIbz/BPr32LsBVDAPBgNVHRMBAf8EBTADAQH/MA4GA1Ud
      DwEB/wQEAwIBBjANBgkqhkiG9w0BAQsFAAOCAQEAaFyLO5h6r1IhR12YvnQw/9Em
      4OVQTpgZuiAY315tBDimk68ACgXo3wSH640OJpuk6w3lQWALxWqBZNCgGHAoR6fE
      zdm13cZaIeWTEhNqjGThlOBo5LoQTX7KEokuLynXxvH55pn44HC9fiYzqmTbak82
      NxtvOIEz/Z+lkyD7HNhZ9FqSfI8DW3cgHe1A8R5EkTrQ8ix3B8A1DRtXABIk47Or
      WTWr4KWVEQMkXjxvuxGGxrAjMpikI8RFHXJkdUbOZPst6hfYZFFHn2g0u96gvUL4
      HcWq3XloiR0+blm+P98yFhdcT9C7dMM6mKOkR9sQALYdLxivTpJhN9vy43lKrA==
      -----END CERTIFICATE-----
```

On configure le nouvel output du serveur initialisé précédemment et on indique l'IP ainsi que le port 9200 et le retour de make fgprint :



Il ne faut pas oublier également de cocher les deux cases puis de mettre le certificat dans la configuration dans la partie yaml :

Maintenant on a notre serveur et notre output :

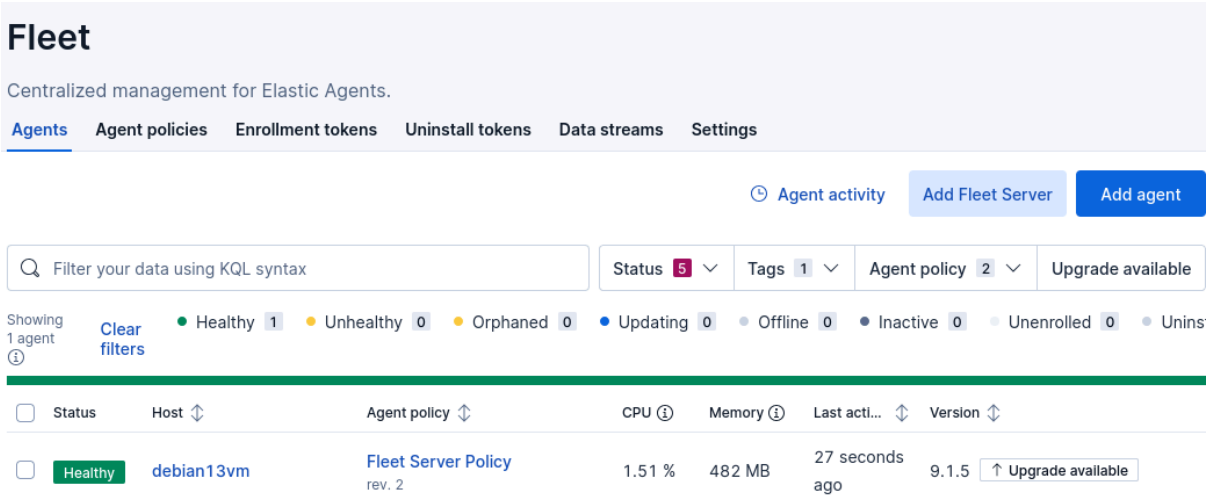


Puis on créé l'agent fleet pour pouvoir déployer les agents sur les vms plus tard avec :

curl -L -O
https://artifacts.elastic.co/downloads/beats/elastic-agent/elastic-agent-9.1.5-linux-arm64.tar.gz
tar xzvf elastic-agent-9.1.5-linux-arm64.tar.gz
cd elastic-agent-9.1.5-linux-arm64
**sudo ./elastic-agent install **
**--fleet-server-es=https://10.203.19.70:9200 **

**--fleet-server-service-token=AAEAAWVsYXN0aWMvZmxlZXQtc2VydmVyL3Rva2VuLTE3NjkxNzY5NjQwMzY6YlpjV1MwSlVTM3lZQ0lOalJSdm5vZw **
**--fleet-server-policy=fleet-server-policy **

**--fleet-server-es-ca-trusted-fingerprint=5A8282D5E401A7434357B394D88D18A30D17C70D1CB07DC6859656EE0E069BCE **
**--fleet-server-port=8220 **
--install-servers
–insecure



3. Déploiement des agents Elastic sur les postes Windows GOAD

a) Automatisation des 3 machines GOAD joignable via les playbooks ansible

Notre objectif est d'automatiser l'installation et l'enrôlement des Elastic Agents sur les machines Windows du domaine GOAD à l'aide d'Ansible, pour qu'elles envoient leurs données vers la stack Elastic.

Pour cela, on commence par installer ansible sur la vm Elastic avec :

sudo apt update
sudo apt install ansible -y
ansible --version

```
root@debian13vm:~/siem# ansible --version
ansible [core 2.18.8]
  config file = /etc/ansible/ansible.cfg
  configured module search path = ['/root/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /root/.local/share/pipx/venvs/ansible/lib/python3.13/site-packages/ansible
  ansible collection location = /root/.ansible/collections:/usr/share/ansible/collections
  executable location = /root/.local/bin/ansible
  python version = 3.13.5 (main, Jun 25 2025, 18:55:22) [GCC 14.2.0] (/root/.local/share/pipx/venvs/ansible/bin/python)
  jinja version = 3.1.6
  libyaml = True
```

J'ai créé le fichier "inventory.ini" afin de référencer les machines Windows GOAD joignables via WinRM, et leur associer des variables de connexion spécifiques.

```
  GNU nano 8.4                                                    inventory.ini
[goad]
GOAD-1 ansible_host=10.203.19.61
GOAD-2 ansible_host=10.203.19.62
GOAD-3 ansible_host=10.203.19.63
GOAD-4 ansible_host=10.203.19.64
GOAD-5 ansible_host=10.203.19.64

[goad:vars]
ansible_connection=winrm
ansible_port=5985
ansible_winrm_transport=ntlm
ansible_winrm_server_cert_validation=ignore
ansible_user=vagrant
ansible_password=vagrant
```

Puis on lance la commande **ansible -i inventory.ini goad -m ansible.windows.win_ping** qui va nous permettre de tester que la connexion WinRM fonctionne bien avec chaque machine référencée.

```
root@debian13vm:~# ansible -i inventory.ini goad -m ansible.windows.win_ping
GOAD-DC1 | UNREACHABLE! => {
    "changed": false,
    "msg": "ntlm: the specified credentials were rejected by the server",
    "unreachable": true
}
GOAD-DC3 | UNREACHABLE! => {
    "changed": false,
    "msg": "ntlm: HTTPConnectionPool(host='10.203.19.63', port=5985): Max retries exceeded with url: /wsman (Caused
by NewConnectionError(\"HTTPConnection(host='10.203.19.63', port=5985): Failed to establish a new connection: [Errno
 111] Connexion refusée\"))",
    "unreachable": true
}
GOAD-SRV2 | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
GOAD-DC2 | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
GOAD-SRV3 | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
```

On a eu 2 machines windows (DC1 et DC3) qui ont crash à cause le tour, donc on a perdu l'interface graphique. On a décidé d'utilisé un playbook ansible pour les 3 machines qu'on a rrive à joindre via Winrm, en revanche les deux autres, on configura les agents à la main.

Donc on re configure **inventory.ini** avec que les vms qu'on arrive à joindre

```
  GNU nano 8.4                                          inventory.ini
[goad_operational]
GOAD-SRV2 ansible_host=10.203.19.62
GOAD-SRV3 ansible_host=10.203.19.61
GOAD-DC2 ansible_host=10.203.19.64

[goad_operational:vars]
ansible_user=Vagrant
ansible_password=vagrant
ansible_connection=winrm
ansible_port=5985
ansible_winrm_transport=ntlm
ansible_winrm_server_cert_validation=ignore
```

Puis on créé le playbook dans le fichier **install_elastic_agent.yml :**

```
- name: Install Elastic Agent (Fleet) on GOAD Windows machines
  hosts: goad_operational
  gather_facts: no
  vars:
    fleet_url: "https://10.203.19.70:8220"
```

```
    enrollment_token:
"Rzc0eS1ac0J5czJ1MkVCSlJkOUU6UVd0YmJCVzV1QXgtSnZKcGNOOUtZdw=="
    elastic_agent_version: "9.1.9"
    zip_url: "https://artifacts.elastic.co/downloads/beats/elastic-agent/elastic-agent-{{
elastic_agent_version }}-windows-x86_64.zip"
    zip_dest: "C:\\Windows\\Temp\\elastic-agent-{{ elastic_agent_version }}.zip"
    extract_base: "C:\\Windows\\Temp\\elastic-agent"
    extracted_dir: "{{ extract_base }}\\elastic-agent-{{ elastic_agent_version
}}-windows-x86_64"

  tasks:
    - name: Check if Elastic Agent service already exists
      ansible.windows.win_shell: |
        if (Get-Service -Name 'Elastic Agent' -ErrorAction SilentlyContinue) { 'present' } else {
'absent' }
      register: agent_service
      changed_when: false

    - name: Allow outbound Fleet 8220 (firewall rule)
      ansible.windows.win_shell: |
        if (-not (Get-NetFirewallRule -DisplayName 'Allow Fleet 8220' -ErrorAction
SilentlyContinue)) {
          New-NetFirewallRule -DisplayName 'Allow Fleet 8220' -Direction Outbound -Action
Allow -Protocol TCP -RemoteAddress 10.203.19.70 -RemotePort 8220
        }
      when: agent_service.stdout.find('absent') != -1

    - name: Download Elastic Agent zip
      ansible.windows.win_get_url:
        url: "{{ zip_url }}"
        dest: "{{ zip_dest }}"
      when: agent_service.stdout.find('absent') != -1

    - name: Create extraction folder
      ansible.windows.win_file:
        path: "{{ extract_base }}"
        state: directory
      when: agent_service.stdout.find('absent') != -1

    - name: Unzip Elastic Agent
      community.windows.win_unzip:
        src: "{{ zip_dest }}"
        dest: "{{ extract_base }}"
      when: agent_service.stdout.find('absent') != -1

    - name: Install and enroll Elastic Agent to Fleet
      ansible.windows.win_shell: |
        cd '{{ extracted_dir }}'
```

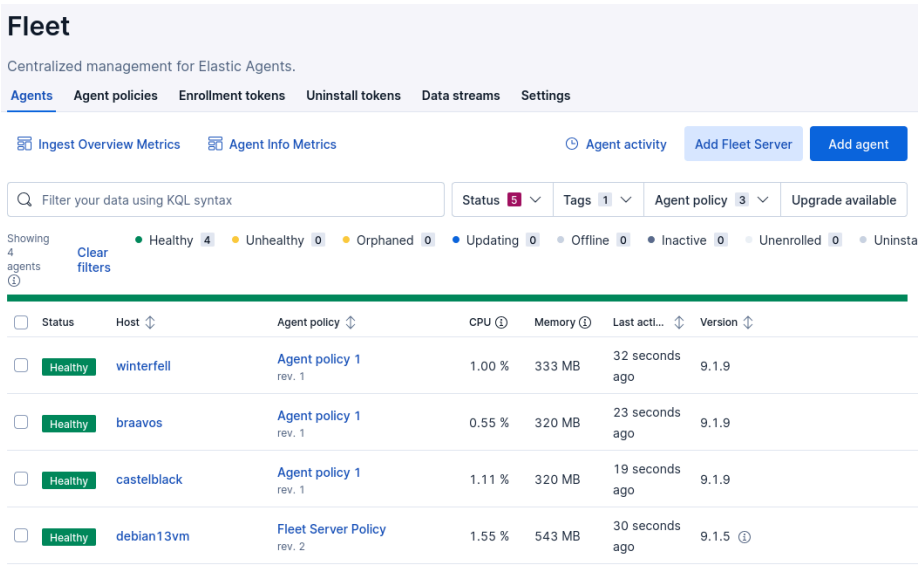
```
      .\elastic-agent.exe install --url={{ fleet_url }} --enrollment-token={{ enrollment_token }}
--insecure --non-interactive --force
    args:
      chdir: "{{ extracted_dir }}"
    when: agent_service.stdout.find('absent') != -1

  - name: Ensure Elastic Agent service is running
    ansible.windows.win_service:
      name: "Elastic Agent"
      start_mode: auto
      state: started
```

Puis on exécute le playbook avec la commande : **ansible-playbook -i inventory.ini install_elastic_agent.yml --limit goad_operational**

On peut voir sur la capture ci dessus que l'installation et l'enrôlement automatique des agents Elastic sur les 3 machines joignables se sont déroulés avec succès.

```
TASK [Install and enroll Elastic Agent to Fleet] ********************************************************
skipping: [GOAD-SRV2]
changed: [GOAD-SRV3]
changed: [GOAD-DC2]

TASK [Ensure Elastic Agent service is running] **********************************************************
ok: [GOAD-SRV2]
ok: [GOAD-DC2]
changed: [GOAD-SRV3]

PLAY RECAP **********************************************************************************************
GOAD-DC2                   : ok=7    changed=5    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0

GOAD-SRV2                  : ok=3    changed=1    unreachable=0    failed=0    skipped=5    rescued=0    ignored=0

GOAD-SRV3                  : ok=7    changed=6    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0
```

Chaque agent apparaît bien dans Kibana sous Fleet → Agents avec l'état "Healthy", la bonne politique ("Agent policy 1") et la version 9.1.9.



b) Configuration manuelle des deux autres machines

Pour les machines GOAD-DC1 et GOAD-DC3 qui ne répondaient plus via WinRM, nous avons procédé à une installation manuelle de l'agent Elastic :

On commence par aller dans **Fleet > Add agent** puis on choisis la policy "Agent Policy" puis on choisis Windows dans la liste déroulante .

Elastic nous génère la commande PowerShell complète :

$ProgressPreference = 'SilentlyContinue'
Invoke-WebRequest -Uri https://artifacts.elastic.co/downloads/beats/elastic-agent/elastic-agent-9.1.9-windows-x86_64.zip -OutFile elastic-agent-9.1.9-windows-x86_64.zip
Expand-Archive .\elastic-agent-9.1.9-windows-x86_64.zip -DestinationPath .
cd elastic-agent-9.1.9-windows-x86_64
.\elastic-agent.exe install --url=https://10.203.19.70:8220 --enrollment-token=Rzc0eS1ac0J5czJ1MkVCSlJkOUU6UVd0YmJCVzV1QXgtSnZKcGNOOUtZdw==

On retrouve els deux autres machines :

Status	Host	Agent policy	CPU	Memory	Last acti...	Version
Healthy	kingslanding	Agent policy 1 rev. 1	0.96 %	406 MB	17 seconds ago	9.1.9
Healthy	meereen	Agent policy 1 rev. 1	0.66 %	394 MB	25 seconds ago	9.1.9
Healthy	winterfell	Agent policy 1 rev. 1	0.92 %	349 MB	30 seconds ago	9.1.9
Healthy	braavos	Agent policy 1 rev. 1	0.51 %	333 MB	18 seconds ago	9.1.9
Healthy	castelblack	Agent policy 1 rev. 1	1.07 %	333 MB	9 seconds ago	9.1.9